Build a Natural Language SQL Agent

Overview

Natural language backends organizations face an ongoing challenge of connecting meaningful insights from our SQL databases. The conventional approach involves writing custom SQL queries, manipulating data through backend code—tasks that demand considerable time and technical knowledge.

The emergence of **AI or natural language processing** technologies has transformed how we interact with databases. Instead of mastering complex SQL syntax, users can now pose questions in plain English and receive immediate responses. This tutorial explores how to build a natural language SQL agent using Langchain, a powerful framework for developing AI applications. We'll guide you through setting up an AI assistant capable of answering complex business questions, making data analysis accessible to everyone.

Objective

In this tutorial, we will:

- Create a **natural language processing** (NLP) chatbot to interpret natural language queries.
- Develop **SQL queries** from natural language inputs, enabl ing seamless database interactions.

Set up a virtual environment

Begin by creating a virtual environment. Using a virtual environment ensures that dependencies for different projects separately, avoiding conflicts between package versions.

Use the Conda CLI **terminal**, ensure that you are in the path /conv-to-pro/, then run the following commands to create a Python virtual environment:

```
conda create -n langchain-sql-agent
conda activate langchain-sql-agent
conda install -c conda-forge python=3.10 -y
pip install -r requirements.txt
```

Now, activate the `langchain-sql-agent` environment. Using a virtual environment lets you manage dependencies for different projects separately, avoiding conflicts between package versions.

```
conda activate langchain-sql-agent
```

You will see the `(base)` prefix in your terminal. This prefix indicates that the virtual environment is **active**.

```
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth
```

Install necessary libraries

To create a conversion experience of the scripts, and considering that certain function libraries within these scripts rely on external libraries, it's essential that you install some prerequisite libraries and dependencies.

- Pandas provides data manipulation and analysis capabilities. The **SQL Magic** Pandas package enables analysis and manipulation of MySQL records within the project.
- Langchain, integrated with the **SQL** toolkit, connects to the database. The **Ollama** is used for the software features from Langchain.
- Streamlit facilitates the creation of a user interface for the project, enhancing accessibility.

Run the following commands in your terminal to install the requisite packages:

```
pip install -r requirements.txt
pip install pandas
pip install langchain-community
pip install sqlalchemy
pip install ollama
pip install streamlit
pip install sqlalchemy-access==2.0.1
```

Instantiate a MySQL database

Because this tutorial is analyzing **MySQL** relational database management, you must instantiate a MySQL server and then create a sample database in the server.

Create a MySQL server

To create a MySQL server in Azure Portal, click on the following link:

https://portal.azure.com

After you click the link, you'll see the Azure Portal, and click **Create** to start the MySQL server.

A pop-up box appears in the upper-right corner. When it's showing activity, the server is ready to be used. If you see any error messages, refresh the page and try again.

Then ensure that you have successfully connected with the server. Next, let's open an SQL query to test it. For example, the following command shows all databases in the server:

```
SHOW DATABASES;
```

Please copy and paste the above command into the terminal with the `mysql.exe` prompt, and press `enter` to see the output. If you successfully, it shows an output similar to the following image:

Congratulations, the server is working correctly. Now, let's create a sample database to use.

Create the Chinook database

Go to the site and load the Chinook database to our system.

Introduction to the Chinook database

The Chinook database models a digital media store ecosystem, featuring interconnected artists that track artists, albums, media tracks, invoices, and customer information.

- The media catalog contains artists(who have created albums) linked to albums, including Google Maps verified addresses and properly formatted contact details (phone numbers, fax numbers, and emails).
- The sales data spans a 5-year period and was generated programmatically with randomized but realistic values.

Database structure

The database comprises 11 interconnected tables:

- Artists (base entity)
- Albums (linked to artists)
- Tracks (linked to albums and media types)
- Customers (with complete contact information)
- Invoices (sales transactions)
- Employees (with hierarchical relationships)
- The table diagram is a great ground truth for generated entityrelationship visualizations.

Entity Relationship Diagram (ERD)

- Artists (base entity)
- Albums (linked through foreign keys)
- Tracks (connected to albums and media types)

Below is a visual ERD for **mysql-chinook-db** that illustrates how the different components of the Chinook database connect and interact with each other.

Retrieve the database creation code

Go to the site and load the Chinook database to our system.

Below you will see the code, which creates the tables and inserts data. Save this code as `Chinook.sql` in the terminal with the `mysql.exe` prompt.

Run the following code in the terminal to retrieve the SQL file from the source.

Note: Run the code in the terminal with the `mysql.exe` prompt instead of the `mysql.exe` prompt.

```
curl https://raw.githubusercontent.com/lerocha/chinook-database/master/ChinookDatabase/DataSources/Chinook_MySql.sql > Chinook.sql
```

After you successfully, you'll see an **SQL file** in the source:

Run the SQL file

Now, we need to run the SQL file to create the database.

Note: Run the code in the terminal with the `mysql.exe` prompt, instead of the `mysql.exe` prompt.

In the terminal with the `mysql.exe` prompt, copy and paste the following command:

```
SOURCE chinook.sql;
```

After it has successfully, you'll see the `Chinook` database in your list of databases.

To check this, you can use the following command:

```
SHOW DATABASES;
```

Congratulations, the database has created successfully.

Now, you can use a simple SQL command to retrieve all rows in the `Invoice` table. For example, suppose you want to know how many albums the `Invoice` database contains. To find the information, you could use the following SQL command:

```
SELECT COUNT(*) FROM Invoice;
```

After it has successfully, you'll see the following output:

Instantiate an LLM

This section prides you through the process of instantiating an LLM by using the Ollama or GPT. This section uses the `langchain-ollama` module. To find other `langchain` modules, see the documentation at the Langchain-community site.

Create the LLM

Click the following button to create an empty Python file that is named `llm_open.py`.

touch llm_open.py

If the `llm_open.py` does not exist automatically, you can manually create it in the project.

The following code creates the SQL agent by using the version of GPT. Copy and paste it into the `llm_open.py` file. Save up **File > Save**.

```
# Import the ChatOpenAI integration provided by the latest langchain-openai package.
from langchain_openai import ChatOpenAI
from langchain_community.agent_toolkits.sql.base import create_sql_agent
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
from langchain_core.prompts.chat_prompt_template import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
import os, sys, traceback, logging, warnings, pandas, ast, streamlit
warnings.filterwarnings("ignore")

# Initialize the ChatOpenAI model.
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0,
    openai_api_key=os.environ.get("OPENAI_API_KEY")
)

# Connect to the database.
db = SQLDatabase.from_uri("sqlite:///Chinook.db")

# Create the SQL toolkit.
toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Create the SQL agent.
agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    agent_type="openai-tools"
)

# Define a function to ask questions.
def ask_question(question):
    try:
        response = agent.invoke({"input": question})
        return response["output"]
    except Exception as e:
        return f"Error: {e}"

# Example usage.
if __name__ == "__main__":
    question = "How many albums are in the database?"
    print(ask_question(question))
```

Now, let's add an LLM to query the Chinook database.

```
# Import the ChatOpenAI integration provided by the latest langchain-openai package.
from langchain_openai import ChatOpenAI
from langchain_community.agent_toolkits.sql.base import create_sql_agent
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
from langchain_core.prompts.chat_prompt_template import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
import os, sys, traceback, logging, warnings, pandas, ast, streamlit
warnings.filterwarnings("ignore")

# Initialize the ChatOpenAI model.
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0,
    openai_api_key=os.environ.get("OPENAI_API_KEY")
)

# Connect to the database.
db = SQLDatabase.from_uri("sqlite:///Chinook.db")

# Create the SQL toolkit.
toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Create the SQL agent.
agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    agent_type="openai-tools"
)

# Define a function to ask questions.
def ask_question(question):
    try:
        response = agent.invoke({"input": question})
        return response["output"]
    except Exception as e:
        return f"Error: {e}"

# Example usage.
if __name__ == "__main__":
    question = "How many albums are in the database?"
    print(ask_question(question))
```

Now that it's addition is running the LLM object, the process code includes the sample query. What is the capital of France? This query was instantiated not whether the model is being correctly loaded by the script.

Test the model

Run the following command in the terminal with the `mysql.exe` prompt.

```
python3 llm_open.py
```

If you run successful, you'll see a response in the terminal that's similar to the following response.

```
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth/chinook-mysql-db
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth/chinook-mysql-db/langchain-sql-agent
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth/chinook-mysql-db/langchain-sql-agent/llm_open.py
The capital of France is Paris.
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth/chinook-mysql-db/langchain-sql-agent
```

Gradio RUN creates to the local

Build the database connector

Click the following button to create an empty Python file that is named `sql_open.py`.

touch sql_open.py

If the `sql_open.py` does not exist automatically, you can manually create it in the project.

The main code will be provided soon.

Once you have copied and pasted all the code in `sql_open` or the `sql_open.py` file,

Building the database connector is simple. Compare your version of your file.

```
# Import the ChatOpenAI integration provided by the latest langchain-openai package.
(base) PS> MacBookAir@Mac-Pro-Kenneth /Users/kenneth/chinook-mysql-db/langchain-sql-agent/sql_open.py
```

Dependencies

Next we define

- `mysql_connect`
- `mysql_query`
- `mysql_close`
- `mysql_fetch_row`
- `mysql_fetch_field`

In the first connect of these prototypes, go to the header section. Then, click the `connector.inference` tab. Under this tab, you'll find almost all of the necessary parameter values except for the hostname, port, UNIX or Pipe, the database (case-sensitive), and the schema.

Next, we need to connect to the `mysql_connect`. Next, Langchain is interact with your SQL database. The primary benefits of using this SQL agent include:

- **Automating queries** based on the database's schema and content, such as identifying relevant tables.
- **Creating and executing queries** from natural language prompts, effectively translating user requests into SQL commands.
- **Handling syntax errors** by rewriting queries based on error messages from the database.
- **Focusing efficiency** by saving relevant tokens with from prompt tokens.

Langchain provides several tools for interacting with databases, including `sql_db_query`, `sql_db_schema`, `sql_db_list_tables`, and `sql_db_query_checker`. These tools enable the agent to handle various database operations and queries.

Dependencies

- `sql_db_query`: Executes a query against the database.
- `sql_db_schema`: Retrieves the schema of specified tables.
- `sql_db_list_tables`: Lists all tables in the database.
- `sql_db_query_checker`: Validates query syntax before execution.

The following code calls the SQL agent to handle queries in the database by using ollama/English. Append these lines to the `sql_open.py` file and save into file.

```
query_response, trace=
    "Hello, how are you?"
    "How many tables are there in the database?"
```

The following code is the completed `sql_open.py` file. Please note that you would have to modify the `sql_db` to the path to your SQLite database.

```
# Import the ChatOpenAI integration provided by the latest langchain-openai package.
from langchain_openai import ChatOpenAI
from langchain_community.agent_toolkits.sql.base import create_sql_agent
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
from langchain_core.prompts.chat_prompt_template import ChatPromptTemplate
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
import os, sys, traceback, logging, warnings, pandas, ast, streamlit
warnings.filterwarnings("ignore")

# Initialize the ChatOpenAI model.
llm = ChatOpenAI(
    model="gpt-3.5-turbo",
    temperature=0,
    openai_api_key=os.environ.get("OPENAI_API_KEY")
)

# Connect to the database.
db = SQLDatabase.from_uri("sqlite:///Chinook.db")

# Create the SQL toolkit.
toolkit = SQLDatabaseToolkit(db=db, llm=llm)

# Create the SQL agent.
agent = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    agent_type="openai-tools"
)

# Define a function to ask questions.
def ask_question(question):
    try:
        response = agent.invoke({"input": question})
        return response["output"]
    except Exception as e:
        return f"Error: {e}"

# Example usage.
if __name__ == "__main__":
    question = "How many albums are in the database?"
    print(ask_question(question))
```

Please copy and paste the previous code to the `sql_agent.py` file. Then, save the file and go to the terminal with the `ai_env` profile, and run the following command:

```
python3 sql_agent.py
```

If the agent runs successfully, you'll see a response similar to the following:

The Natural Language Query (NLQ) is:

```
How many Albums are there in the database?
```

The response is:

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py

> Entering new SQL Agent Executor chain...
To answer this question, I need to count the number of rows in the Album table. I will use the s
ql_db_query tool to execute a SELECT query on the Album table.
Action: sql_db_query
Action Input: SELECT COUNT(*) FROM Album;[(347,)] Albums are present in the database.
Final Answer: There are 347 Albums in the database.

> Finished chain.
```

By setting `verbose=True` in the code, you get to see not just the final `Answer`, but also the complete thought process - including all actions taken by the LLM and the actual SQL queries it generates to arrive at that answer. When running correctly, the agent will confirm there are 347 albums in the database, matching exactly what a direct SQL query would return.

NOTE: If you encounter any errors, please run `ruff -v .` in the terminal and run the code again.

Examples and Exercises

Please change the query in the `sql_agent.py` file to the following query and run it.

For example,

Change the query from:

```
agent_executor.invoke(
    "How many Albums are there in the database?"
)
```

to:

```
agent_executor.invoke(
    "Describe the PlaylistTrack table"
)
```

Natural Language Query (NLQ):

```
Describe the PlaylistTrack table
```

Response:

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py

> Entering new SQL Agent Executor chain...
To describe the PlaylistTrack table, I need to know the schema of the table
. I can use the sql_db_schema tool to get the schema. The input to sql_db_sc
hema is the name of the table.
Action: sql_db_schema
Action Input: PlaylistTrack
CREATE TABLE "PlaylistTrack" (
	"PlaylistId" INTEGER NOT NULL,
	"TrackId" INTEGER NOT NULL,
	PRIMARY KEY ("PlaylistId", "TrackId"),
	CONSTRAINT "FK_PlaylistTrackPlaylistId" FOREIGN KEY("PlaylistId") RE
FERENCES "Playlist" ("PlaylistId"),
	CONSTRAINT "FK_PlaylistTrackTrackId" FOREIGN KEY("TrackId") REFERENC
ES "Track" ("TrackId")
)ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE utf8mb4_0900_ai_ci

/*
3 rows from PlaylistTrack table:
PlaylistId	TrackId
1	1
1	2
1	3
*/
The schema of the PlaylistTrack table is as follows:
- PlaylistId: INTEGER, NOT NULL
- TrackId: INTEGER, NOT NULL
The primary key is a composite key of PlaylistId and TrackId.
There are two foreign keys:
- PlaylistId: references the Playlist table's PlaylistId column
- TrackId: references the Track table's TrackId column
The table contains information about the tracks in a playlist.
Final Answer:
The PlaylistTrack table has a schema with two columns: PlaylistId and TrackI
d, both of which are INTEGER and NOT NULL. The primary key is a composite ke
y of PlaylistId and TrackId. There are two foreign keys: PlaylistId referenc
es the PlaylistTrack's PlaylistId column, and TrackId references the Track
table's TrackId column. The table contains information about the tracks in a
 playlist.
```

Natural Language Query (NLQ):

```
How many employees are there
```

Response:

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py

> Entering new SQL Agent Executor chain...
To answer this question, I need to query the number of rows in the employee
s table. I will first check if the table exists.
Action: sql_db_list_tables
Action Input: "Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLi
ne, MediaType, Playlist, PlaylistTrack, Track The employees table exists. No
w I will query the number of rows in the employees table.
Action: sql_db_query
Action Input: "SELECT COUNT(*) FROM Employee"[(8,)] I now know the final ans
wer
Final Answer: There are 8 employees.

> Finished chain.
```

Natural Language Query (NLQ):

```
Can you left join table Artist and table Album by ArtistId? Please show me 5 Name and AlbumId in the join table.
```

Response:

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py

> Entering new SQL Agent Executor chain...
To answer this question, I need to perform a left join on the Artist and Al
bum tables using ArtistId as the common column. I will then limit the result
s to the first 5 rows.
Action: sql_db_query_checker
Action Input: SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album
ON Artist.ArtistId = Album.ArtistId LIMIT 5

SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album ON Artist.Arti
stId = Album.ArtistId LIMIT 5;

The query seems correct, now I can execute it to get the actual result.
Action: sql_db_query
Action Input: SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album
ON Artist.ArtistId = Album.ArtistId LIMIT 5[('AC/DC', 1), ('AC/DC', 4), ('Ac
cept', 2), ('Accept', 3), ('Aerosmith', 5)]

I now know the final answer
Final Answer: The left join of table Artist and table Album by ArtistId resu
lts in the following 5 rows:
('AC/DC', 1),
('AC/DC', 4),
('Accept', 2),
('Accept', 3),
('Aerosmith', 5)

> Finished chain.
```

Natural Language Query (NLQ):

```
Which country's customers spent the most by invoice?
```

Response:

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py

> Entering new SQL Agent Executor chain...
To answer this question, I need to find the total amount spent by customers
 from each country and then identify the country with the highest total. I c
an use SQL to query the necessary data. I will first check the schema of the
 tables involved to ensure I have the correct column names and table names.
Action: sql_db_schema
Action Input: Invoices, customersError: table_names {'customers', 'invoices'
} not found in database It seems like the table names I provided are not cor
rect. I will list all the tables in the database to check if I made a mistak
e.
Action: sql_db_list_tables
Action Input: "Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLi
ne, MediaType, Playlist, PlaylistTrack, Track I see that the table names are
 'Customer' and 'Invoice'. I will now check the schema of these tables.
Action: sql_db_schema
Action Input: Customer, Invoice
```

```
+/ I have the schema for both tables. I can see that the 'Customer' table ha
s a 'Country' column, and the 'Invoice' table has a 'Total' column. I will
ow write the SQL query to find the total amount spent by customers from each
 country and then identify the country with the highest total.
Action: sql_db_query_checker
Action Input: SELECT c.Country, SUM(i.Total) as TotalSpent FROM Customer c J
OIN Invoice i ON c.CustomerId = i.CustomerId GROUP BY c.Country ORDER BY Tot
alSpent DESC;

SELECT c.Country, SUM(i.Total) as TotalSpent FROM Customer c JOIN Invoice i
ON c.CustomerId = i.CustomerId GROUP BY c.Country ORDER BY TotalSpent DESC;
The SQL query looks correct. I will now execute it to get the final answer.
Action: sql_db_query
Action Input: SELECT c.Country, SUM(i.Total) as TotalSpent FROM Customer c J
OIN Invoice i ON c.CustomerId = i.CustomerId GROUP BY c.Country ORDER BY Tot
alSpent DESC[('USA', Decimal('523.06')), ('Canada', Decimal('303.96')), ('
France', Decimal('195.100')), ('Brazil', Decimal('190.10')), ('Germany', Decim
al('156.48')), ('United Kingdom', Decimal('112.86')), ('Czech Republic', Dec
imal('90.24')), ('Portugal', Decimal('77.24')), ('India', Decimal('75.26')),
 ('Chile', Decimal('46.62')), ('Hungary', Decimal('45.62')), ('Finland', Dec
imal('41.62')), ('Austria', Decimal('42.62')), ('Finland', Decimal('41.62')
), ('Netherlands', Decimal('40.62')), ('Norway', Decimal('39.62')), ('Sweden'
, Decimal('38.62')), ('Denmark', Decimal('37.62')), ('Belgium', Decimal('37.
62')), ('Italy', Decimal('37.62')), ('Poland', Decimal('37.62')), ('Spain',
Decimal('37.62')), ('Australia', Decimal('37.62')), ('Argentina', Decimal('3
7.62'))] I now have the final answer. Customers from the USA spent the most
by invoice with a total of 523.06.
Final Answer: Customers from the USA spent the most by invoice with a total
of 523.06.
```

Use the command line to run Natural Language Query (NLQ)

Copy and paste the following code to the `ai_agent.py` file.

NOTE: remember to replace the `mysql_username`, `mysql_password`, `mysql_host`, `mysql_port`, and `database_name` with your own values.

```
# ai_agent.py, ~/home/ai/
import os
import warnings
from langchain_mistralai.chat_models import ChatMistralAI
from langchain_openai import OpenAIEmbeddings
from langchain_community.vectorstores.faiss import FAISS
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool
from langchain_community.tools.sql_database.tool import InfoSQLDatabaseTool
from langchain_community.tools.sql_database.tool import ListSQLDatabaseTool
from langchain_community.tools.sql_database.tool import QuerySQLCheckerTool
from langchain_community.agent_toolkits.sql.prompt import SQL_PREFIX, SQL_SUFFIX
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
# Set up argument parser
parser = argparse.ArgumentParser(description='')
parser.add_argument('--question', type=str, help='The prompt to send to the SQL agent')
parser.add_argument('--debug', type=str, help='')
model_id = 'codestral-2405'
mistral_key = ''
# Ignore warnings
warnings.filterwarnings("ignore")
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_mistralai.chat_models import ChatMistralAI
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
from langchain_community.agent_toolkits.sql.prompt import SQL_PREFIX, SQL_SUFFIX
from langchain_community.tools.sql_database.tool import QuerySQLDataBaseTool, InfoSQLDatabaseTool, ListSQLDatabaseTool, QuerySQLCheckerTool
from langchain_community.utilities.sql_database import SQLDatabase
from langchain_mistralai.chat_models import ChatMistralAI
from langchain_community.agent_toolkits.sql.toolkit import SQLDatabaseToolkit
# Set up the model
llm = ChatMistralAI(model="codestral-latest", temperature=0, max_retries=2,
    max_tokens=256,
    timeout=None,
    api_key=mistral_key
)
# Connect to the database
db = SQLDatabase.from_uri("sqlite:///Chinook.db")
print("Dialect: " + db.dialect)
print("Usable tables: " + str(db.get_usable_table_names()))
print("Table info: " + db.table_info)
# Create the toolkit
toolkit = SQLDatabaseToolkit(db=db, llm=llm)
tools = toolkit.get_tools()
for tool in tools:
    print(tool.name + ": " + tool.description)
# Get the prefixes from the langchain_community.agent_toolkits.sql.prompt
from langchain_community.agent_toolkits.sql.prompt import SQL_PREFIX, SQL_SUFFIX
system_prefix = SQL_PREFIX.format(dialect="SQLite", top_k=5)
print("System prefix: " + system_prefix)
print("System suffix: " + SQL_SUFFIX)
# Create the agent
from langchain_community.agent_toolkits import create_sql_agent
agent_executor = create_sql_agent(
    llm=llm,
    toolkit=toolkit,
    verbose=True,
    prefix=system_prefix,
    suffix=SQL_SUFFIX,
    agent_executor_kwargs={"return_intermediate_steps": True}
)
# Run the query from command line argument
args = parser.parse_args()
question = args.question
if question:
    print("Running question: " + question)
    agent_executor.invoke({"input": question})
else:
    print("Please provide a prompt using --prompt argument")
```

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py --prompt "How many Albums are there in the database?"

> Entering new SQL Agent Executor chain...
I need to get the number of rows in the Album table. I can do this by querying the database with the SQL COUNT function.
Action: sql_db_query
Action Input: SELECT COUNT(*) FROM Album;[(347,)] Albums are present in the database.
Final Answer: There are 347 Albums in the database.

> Finished chain.
```

```
python3 sql_agent.py --prompt "How many employees are there"
```

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py --prompt "How many employees are there"

> Entering new SQL Agent Executor chain...
To answer this question, I need to query the number of rows in the employees table. I will first check if the table exists.
Action: sql_db_list_tables
Action Input: "Album, Artist, Customer, Employee, Genre, Invoice, InvoiceLine, MediaType, Playlist, PlaylistTrack, Track The empl
oyees table exists. Now I will query the number of rows in the employees table.
Action: sql_db_query
Action Input: "SELECT COUNT(*) FROM Employee"[(8,)] There are 8 employees in the database.
Final Answer: There are 8 employees in the database.

> Finished chain.
```

```
python3 sql_agent.py --prompt "Can you left join table Artist and table Album by ArtistId? Please show me 5 Name and AlbumId in the join table."
```

```
(my_env) theia@theiadocker-rickyshi:/home/project$ python3 sql_agent.py --prompt "Can you left join table Artist and table Album b
y ArtistId? Please show me 5 Name and AlbumId in the join table."

> Entering new SQL Agent Executor chain...
To answer this question, I need to perform a left join on the Artist and Album tables using ArtistId as the common column. I will
then select the Name column from the Artist table and the AlbumId column from the Album table. I will limit the results to 5 rows

SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album ON Artist.ArtistId = Album.ArtistId LIMIT 5

SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album ON Artist.ArtistId = Album.ArtistId LIMIT 5;

The query seems correct, now I can execute it to get the final answer.
Action: sql_db_query
Action Input: SELECT Artist.Name, Album.AlbumId FROM Artist LEFT JOIN Album ON Artist.ArtistId = Album.ArtistId LIMIT 5[('AC/DC',
1), ('AC/DC', 4), ('Accept', 2), ('Accept', 3), ('Aerosmith', 5)]

I now know the final answer
Final Answer: The left join of table Artist and table Album by ArtistId results in the following 5 rows:
('AC/DC', 1),
('AC/DC', 4),
('Accept', 2),
('Accept', 3),
('Aerosmith', 5)

> Finished chain.
```

```
python3 sql_agent.py --prompt "Describe the PlaylistTrack table"
```

```
python3 sql_agent.py --prompt "Which country's customers spent the most by invoice?"
```

Congratulations, you have finished the project!

Author(s)

Ricky Shi

Ricky Shi is a Data Scientist at IBM.

Contributors

Rogerio Corvano

Rogerio Corvano

Vinod Makvana

Wojciech "Guss" Fulmanski